

25. Reading and writing binary files

Herman Kamper

In the previous lecture we read and wrote text files. We could open them in a text editor, check by eye that the right thing had been written, and process them line by line. This lecture is about a different kind of file: binary files. They look like a more complicated version of the same thing, but the idea behind them is simple, and it follows directly from asking a question about text files that we skipped: how many bytes does a text file actually take up, and is that what we want?

Stay with this one from the start. Everything depends on keeping straight what is in the computer's memory and what is on the disk.

Writing a text file, and looking at what we wrote

Let us begin with a quick recap. Here is a program that writes a single float to a text file. By now every line of it should be familiar:

```
#include <stdio.h>

int main()
{
    FILE *fPtr;

    float value = 12.34567;

    fPtr = fopen("value.txt", "w");
    fprintf(fPtr, "%f", value);
    fclose(fPtr);

    return 0;
}
```

We open `value.txt` for writing, use `fprintf` exactly as we would use `printf` but with the file handle as the first argument, and close the file. Run it, and a new file called `value.txt` appears next to the program. Open it in a text editor and you see what you would expect:

```
12.345670
```

We could read it back with `fscanf` if we wanted to. Nothing new so far.

Now for the question we have not asked before. How big is `value.txt`? Your operating system

will tell you if you look at the properties of the file, or if you list the directory with a command like `ls -l`. The answer is 9 bytes.

That makes sense once you count the characters in `12.345670`: there are nine of them, and each character takes one byte. So the file contains nine bytes, each of which is eight ones and zeros.

Memory and the hard drive

Before we go on, it helps to be precise about what “writing to a file” means. Everything we did before we met files lived in the computer’s memory, the random access memory (RAM). Memory is made of tiny regions of silicon that are held at a high or a low voltage, and those represent the ones and zeros, which in turn represent our integers, characters and floats. We manipulate them, and when the program ends, or the computer is switched off, they are gone.

When we write to a file we are, for the first time, writing to the machine’s hard drive. The drive stores ones and zeros in a physical form that survives switching off the power. If you turn the computer off and on again, `value.txt` is still sitting there, while the variable `value` is long gone. A file-writing program takes something that was in memory and dumps a copy of it onto permanent storage.

So what exactly is on the disk in `value.txt`? You can look at the ones and zeros with a tool that shows a file’s bytes in binary. On Linux and macOS, `xxd -b value.txt` does it:¹

```
00110001 00110010 00101110 00110011 00110100 00110101 00110110 00110111
                                             00110000
```

That is nine groups of eight bits, one group per byte. Each byte is just a binary number, so we can convert it to ordinary decimal. Take the third one, `00101110`:²

$$00101110_2 = 32 + 8 + 4 + 2 = 46$$

The byte is the number 46. But why 46? The answer is the ASCII table, the standard that assigns a number to each character, which is how a file of numbers can contain text. Here is the part of it that we need:

¹For Computer Programming 143, you do not need to know this tool. It is only used for illustrative purposes here. What you do need to know is the idea that characters are written to a file on the harddrive as bytes.

²For Computer Programming 143, we will not expect you to be able to do binary to decimal conversion.

Code	Char	Code	Char	Code	Char	Code	Char	Code	Char	Code	Char
46	.	57	9	75	K	86	V	103	g	114	r
47	/	65	A	76	L	87	W	104	h	115	s
48	0	66	B	77	M	88	X	105	i	116	t
49	1	67	C	78	N	89	Y	106	j	117	u
50	2	68	D	79	O	90	Z	107	k	118	v
51	3	69	E	80	P	97	a	108	l	119	w
52	4	70	F	81	Q	98	b	109	m	120	x
53	5	71	G	82	R	99	c	110	n	121	y
54	6	72	H	83	S	100	d	111	o	122	z
55	7	73	I	84	T	101	e	112	p		
56	8	74	J	85	U	102	f	113	q		

Code 46 is the full stop, and the third character in 12.345670 is indeed a full stop. The first byte, 00110001, is 49, which the table says is the character 1. Note that this is the character 1, not the value one: the byte holds 49, and the text editor displays it as a 1 because it looks up 49 in the ASCII table. In the same way the second byte is 50, the character 2, and the file as a whole is the nine characters 1, 2, ., 3, 4, 5, 6, 7, 0.

If you appended an a to the file, it would grow to ten bytes, and the tenth byte would be 97, which is 01100001.

All of this is the key point of this lecture, so let us say it once more in a different way. A file is nothing more than a long row of bytes. When a text editor opens a file, it takes every byte, looks it up in the ASCII table and puts that character on the screen. That is why it can display value.txt so nicely. A text file is simply a file in which the bytes were chosen to be sensible characters.

The bits in memory are not the bits on disk

Now forget about files for a moment and ask: how many bytes does it take to store a float? Four. Somewhere in memory, the variable value is a box of four bytes, and in those four bytes is the representation of 12.34567. On the machine used here, those bytes, in the order they sit in memory, are:

```
11011101 10000111 01000101 01000001
```

Now compare that with what is in the text file:

```
00110001 00110010 00101110 00110011 00110100 00110101 00110110 00110111
                                00110000
```

The two are completely different. They are not even the same length. When we called fprintf, C took the four bytes in the box, worked out which characters you would need to write the number down in decimal, 1, 2, ., 3 and so on, and then wrote the ASCII code of each of those characters to the file. The conversion took us from four bytes in memory to nine bytes on disk.

That is what happens every time you write a number to a text file. Because the file holds characters, a number has to be turned into characters first. There is an upside to this: you can open the file in a text editor and see what is in it. There is also a downside, which is that the file is often much bigger than it needs to be.

For one `float` the extra five bytes do not matter. But imagine you have trained a neural network with five million parameters, which is a small network, and you want to save it. As four-byte floats it takes about 20 megabytes in memory. Written with `fprintf`, each number becomes nine or more characters, plus a separator, and the file is more than twice that size. You spent a lot of effort getting those numbers, and more than half of the space they take on disk is overhead from the conversion.

Is there a better way? Yes, and the obvious one works: do not convert anything. The four bytes in memory are already a perfectly good record of the number, so write exactly those four bytes to the disk, and when you need the number back, read those four bytes into a `float` again. Then the file is four bytes long, and the file for the whole network is the same size as the network in memory.

There is a price. If you open this file in a text editor, you will see garbage, because the editor will try to treat those bytes as ASCII codes, and the bytes are not characters. At best you get a few strange symbols, and, if one of the bytes happens to be the code of a letter, you get that letter. We can accept this: we only want to store the numbers and load them again later, and we never planned to read the file by eye.

This is exactly what a binary file is. A text file is a file in which values have first been converted into characters. A binary file is a file in which whatever was in memory is copied to the disk as it is, with no conversion in between.

`fwrite` and `fread`

For binary files, C gives us two functions to replace `fprintf` and `fscanf`: `fwrite` and `fread`. The names say what they do. Whereas `fprintf` formats values as text, `fwrite` takes bytes from a place in memory and copies them into a file. Going the other way, `fread` copies bytes from a file into a place in memory.

The figure shows what we want to happen with our `float`. On the top, in memory, are the four bytes of value (at address 11093, a made-up number as always) and an empty box called `input` somewhere else. At the bottom, on the hard drive, are the nine bytes of the text file that we saw earlier, and next to them a new file, `newfile.bin`, which will hold just four bytes. `fwrite` copies the four bytes of value into the file, and `fread` copies them back out into `input`:

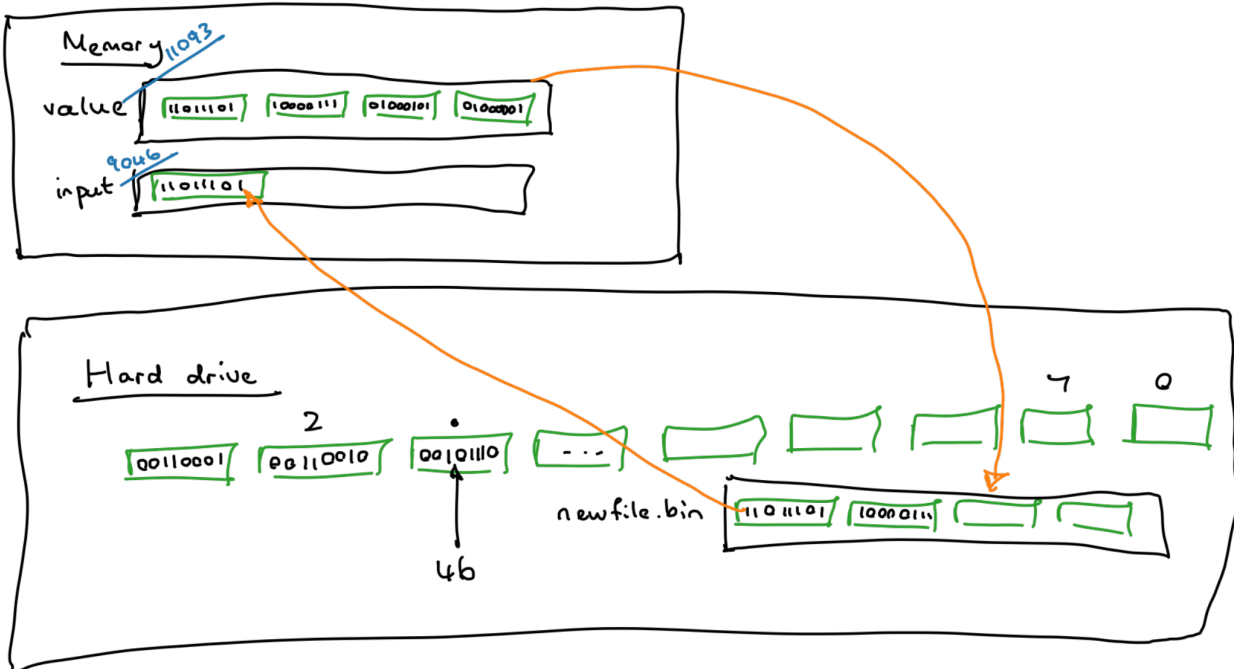


Figure 1: Memory and the hard drive. The four bytes of value are copied by `fwrite` into `newfile.bin` on the drive, and `fread` later copies them back into the box input. The nine bytes of the text file, of which the first two are the characters 1 and 2 (codes 49 and 50) and the third is the full stop (46), are shown for comparison.

Here is the code. First we write the value to a binary file:

```
FILE *binFile;
float value = 12.34567;

binFile = fopen("value.bin", "wb");
fwrite(&value, sizeof(float), 1, binFile);
fclose(binFile);
```

There are two differences from the text-file version. The first is the mode. Instead of "w" we write "wb", which means write, in binary mode. (The file name does not matter to C. Giving it the extension .bin is just a convention.) The second is the call to `fwrite`, which takes four arguments:

1. The address in memory to copy the bytes from. Here that is `&value`, the address of the box.
2. The size, in bytes, of one item. Here that is `sizeof(float)`, which is 4. We could have written 4, but `sizeof(float)` says what we mean, and it stays correct on a machine where a float has a different size.
3. How many items to copy. Here it is 1.
4. The file handle, as always.

The second and third arguments multiply to the total number of bytes that are written: one item

of four bytes, so four bytes. Why are they separate? Because of arrays, as we will see shortly: if you want to write a hundred integers, the size of one item is still `sizeof(int)`, and the count is 100.

The `fwrite` jumps to address 11093, reads four bytes starting there, and puts them into the file. When the program has run, a new file `value.bin` exists, and its size is four bytes. If you look at the bytes with `xxd -b value.bin`, they are exactly the ones and zeros that we saw in memory:

```
11011101 10000111 01000101 01000001
```

If you open it in a text editor you see a few odd symbols, but nothing you can read. That is as expected.

Reading it back is the mirror image. We open the file with `"rb"` for reading in binary mode, make a new box for the result and use `fread`, whose four arguments mean the same as those of `fwrite`, except that the first one is now the place to copy the bytes *into*:

```
float input;

binFile = fopen("value.bin", "rb");
fread(&input, sizeof(float), 1, binFile);
fclose(binFile);

printf("The value is %f\n", input);
```

The output is:

```
The value is 12.345670
```

Four bytes were copied from the file into the box `input`, and, since those bytes were written by a `float` in the first place, they represent the same number again. To put it all in one place, here is the complete program that writes the text file, writes the binary file and reads the binary file back:

```

#include <stdio.h>

int main()
{
    FILE *fPtr;
    FILE *binFile;

    float value = 12.34567;
    float input;

    /* Write text file */
    fPtr = fopen("value.txt", "w");
    fprintf(fPtr, "%f", value);
    fclose(fPtr);

    /* Write binary file */
    binFile = fopen("value.bin", "wb");
    fwrite(&value, sizeof(float), 1, binFile);
    fclose(binFile);

    /* Read binary file */
    binFile = fopen("value.bin", "rb");
    fread(&input, sizeof(float), 1, binFile);
    fclose(binFile);

    printf("The value is %f\n", input);

    return 0;
}

```

After running it, value.txt is nine bytes and value.bin is four.

It is worth pausing on one thing, because it is the most common source of confusion with this topic. A text file and a binary file are both just bytes on a disk. Neither is more of a file than the other. What differs is what the bytes mean and which functions you use to put them there and to read them back. A text editor opens any file and interprets its bytes as ASCII; that happens to be sensible for a text file and is nonsense for a binary one. You use fprintf and fscanf for the first kind and fwrite and fread for the second.

The b in the mode is a separate matter. The functions decide what gets written; the b tells the system not to translate anything on the way. On some systems, text mode translates the newline character on its way to and from the disk, and binary mode does not. Whichever functions you use, open the file with the mode that matches them, so that "wb" goes with fwrite and "w" goes with fprintf.

Writing anything to a binary file

The nice thing about `fwrite` is that it does not care what it is copying. It takes bytes from an address, and you tell it how many. This means you can write anything that is in memory, not only a float.

An array is the simplest case. If `numbers` is an array of 100 integers, then one call writes all of it:

```
fwrite(numbers, sizeof(int), 100, binFile);
```

The array name is already an address, so there is no `&`. The size of one item is `sizeof(int)`, and the count is 100, so 400 bytes are written. The matching `fread` reads them back into an array of the same size:

```
fread(numbers, sizeof(int), 100, binFile);
```

A structure works just as well, and this is where binary files really save effort. Converting a structure to text and back means writing code that deals with each member, one by one. With `fwrite` you copy the whole thing in one go. Given a structure type `struct myStruct` and a variable `myObj` of that type:

```
fwrite(&myObj, sizeof(struct myStruct), 1, binFile);  
fread(&myObj, sizeof(struct myStruct), 1, binFile);
```

And since an array of structures is just an array, an array of a million structures is one `fwrite` and one `fread`:

```
fwrite(myArr, sizeof(struct myStruct), numElements, binFile);  
fread(myArr, sizeof(struct myStruct), numElements, binFile);
```

Let us make this concrete with an example. A triangle in two dimensions is described by the coordinates of its three vertices. We define a structure for a point, with an `x` and a `y`, each a `double`:

```
struct coord  
{  
    double x;  
    double y;  
};  
typedef struct coord Point;
```

The following program asks the user for three vertices and writes each one to a binary file as it is read. Note that this uses the `"wb"` mode and checks that the file opened, as we learnt to do with text files:

```

#include <stdio.h>

struct coord
{
    double x;
    double y;
};
typedef struct coord Point;

int main()
{
    int i;
    Point vertex;
    char *filename = "triangle.bin";
    FILE *fPtr = fopen(filename, "wb");

    if (fPtr == NULL)
    {
        printf("Error!\n");
    }
    else
    {
        for (i = 1; i <= 3; i++)
        {
            printf("Enter x, y for vertex %d: ", i);
            scanf("%lf%lf", &vertex.x, &vertex.y);
            fwrite(&vertex, sizeof(Point), 1, fPtr);
        }
        fclose(fPtr);
    }

    return 0;
}

```

One piece of this is new, though it has nothing to do with files. The `scanf` reads two values at once: with `"%lf%lf"` it reads a double into `vertex.x` and another into `vertex.y`, and the user can type the two numbers separated by a space. A single `scanf` can read as many values as you like in this way.

Suppose we run the program and enter the points (0, 0), (6.9, 0) and (7.7, 9.10101). The program finishes, and we look at the file: it is 48 bytes. We can account for that. A double is 8 bytes, a `Point` holds two of them and so takes 16 bytes, and we wrote three points: $3 \times 16 = 48$.

As before, if you open this file in a text editor you will see only noise.

There is one more thing to see here, and it concerns `fclose`. If you look at the size of the file

while the program is still running, waiting for you to type the second vertex, you will find that the file is empty, even though one vertex has already been passed to `fwrite`. The system collects what you write in memory first and puts it on the disk later, in larger pieces, because that is much faster. Closing the file is what makes sure everything that is waiting actually reaches the disk. If your program crashed before it reached the `fclose`, the last part of your output could be lost. So always close a file you have written to, as early as you can.

Now we read the triangle back. Since the file holds three structures one after another, we read them in the same order into an array of three of them. The `fread` for each vertex copies 16 bytes into the next array element:

```
#include <stdio.h>

struct coord
{
    double x;
    double y;
};
typedef struct coord Coord;

int main(void)
{
    char filename[20];
    FILE *cfPtr;
    int i;
    Coord vertices[3];

    printf("Please enter filename: ");
    scanf("%19s", filename);

    cfPtr = fopen(filename, "rb");

    if (cfPtr == NULL)
    {
        printf("ERROR - file could not be opened!\n");
    }
    else
    {
        for (i = 0; i < 3; i++)
        {
            fread(&vertices[i], sizeof(Coord), 1, cfPtr);
            printf("Vertex %d: (%.11f, %.11f)\n",
                i + 1, vertices[i].x, vertices[i].y);
        }
        fclose(cfPtr);
    }
}
```

```
    return 0;
}
```

Given the file `triangle.bin` from before, the output is:

```
Please enter filename: triangle.bin
Vertex 1: (0.0, 0.0)
Vertex 2: (6.9, 0.0)
Vertex 3: (7.7, 9.1)
```

Just as with a text file, the file handle keeps track of where it is: after each `fread`, it has moved on by the number of bytes that was read, so the second `fread` gets the second vertex. And, as the earlier section suggested, the loop could be replaced by a single call, since the three vertices sit right next to each other in the array, just as they do in the file:

```
fread(vertices, sizeof(Coord), 3, cfPtr);
```

The function `fread` also returns the number of items it managed to read, which would be 3 here. If the file were shorter than expected, the return value would be smaller, which is a way to detect that.

Strings and binary files

One more difference is worth knowing about. Consider a string, `char word[] = "Hello"`. It occupies six bytes in memory: five for the letters and a sixth for the terminating `\0`. If you write it to a text file with `fprintf(fPtr, "%s", word)`, the file contains five bytes, because `fprintf` writes the characters and leaves the `\0` out. With `fwrite`, however, you decide how many bytes to copy. If you ask for `strlen(word) + 1` bytes, you get the six of them, and the `\0` goes into the file as well.

That is one more way of seeing the difference: a text function writes a representation of the value that is suitable for people, whereas `fwrite` copies bytes without asking what they mean.

A word of caution

Copying memory to the disk is powerful, but the file only makes sense to a program that knows what the bytes are. The file `triangle.bin` is just 48 bytes. Nothing in it says that these are three pairs of double values. A program that reads it needs the same structure definition as the one that wrote it, so if the writer and the reader disagree, the reader gets nonsense, and C will not warn you.

Two further things can go wrong. First, the layout of data in memory can differ between machines. The size of an `int`, or the order in which the bytes of a number are stored, is not the same everywhere, so a binary file written on one kind of computer is not guaranteed to read correctly on another. Text files do not have this problem, which is one reason they are still so popular for data that has to be shared. Second, a structure that contains a pointer, such as a `char *name`, cannot be saved meaningfully with a single `fwrite`. What is copied is the address

stored in the pointer, not the characters it points at, and that address means nothing in a later run of the program. For a structure like that, you would store the text it refers to separately, for instance in a character array inside the structure.

None of this makes binary files a bad idea. It just means you should use them when the same program, or a program written by you with the same definitions, will be reading the data back. That is exactly the situation for saving the parameters of a model, or the records of a program, and so on.

Text files versus binary files

We can now put the two kinds of file side by side. In both, what is on the disk is just bytes. What changes is whether C converts your values to characters on the way:

	Text files	Binary files
Write	<code>fprintf(...)</code>	<code>fwrite(...)</code>
Read	<code>fscanf(...)</code>	<code>fread(...)</code>
Mode	"r", "w", "a"	"rb", "wb", "ab"
What is stored	values converted to characters	the bytes that were in memory
In a text editor	readable	looks like noise
Size	whatever the characters need	exactly the size of the data

Which one you pick depends on what you need. If a human has to read the file, or another program has to read it without knowing your structures, use a text file. If you want to store memory exactly and compactly, and the same kind of program will read it, use a binary file.

Summary

A file is a sequence of bytes on the disk. A text editor shows those bytes as ASCII characters; that is sensible for a text file and meaningless for a binary file.

- A text file stores values converted to characters. Writing a float as text turns four bytes in memory into nine or more bytes on disk. It can be read in an editor, but it is often larger than necessary.
- A binary file stores whatever bytes are in memory, with no conversion. A float takes four bytes on disk and four in memory.
- Open a binary file with "rb", "wb" or "ab".
- `fwrite(address, size, count, file)` copies `size * count` bytes from memory at `address` into the file.
- `fread(address, size, count, file)` copies `size * count` bytes from the file into memory at `address`, and returns the number of items it read.
- Anything in memory can be written like this: a single variable, an array, a structure or an array of structures. For an array, the array name is already the address.
- Always close a file you have written, so that everything held back actually reaches the disk.

- A binary file is only meaningful to a program that knows what the bytes are, and it is not guaranteed to move between different kinds of machine.

Looking forward

So far we have dealt with files in a linear sequence: in both text and binary files, we wrote or read to files from the start of the file to the end in a sequential order. In the next lecture we will look at how we can jump to arbitrary points in a binary file and read or write from those points.

Exercises

1. Write a program that writes the integer 1000000 to a text file with `fprintf` and to a binary file with `fwrite`. Compare the sizes of the two files, and explain each. Then view both files with `xxd -b` (or a similar tool) and identify, in the text file, the byte for each digit using the ASCII table.³
2. A text file contains the characters 255. How many bytes does it take? Write the value 255 as an `int` to a binary file and say how many bytes you expect. What would change if you used a `char` instead of an `int`? (A `char` holds one byte.)
3. Write a program that asks the user for 10 integers, stores them in an array and writes the whole array to a binary file with a single call to `fwrite`. Then write a second program that reads the file back into a new array with a single `fread` and prints the numbers. Check the size of the file.
4. Define a structure `struct student` with a name stored in a character array of length 30 and a mark stored as a `double`. Write an array of three students to a binary file and read it back into a different array. How large is the file? Compare with `3 * sizeof(struct student)`.
5. Change your solution to the question above so that the name is a `char *` pointing to a string literal. Write the structures to the file, and then read them back in a *new run* of the program. What do you find, and why?

³You can use <https://hexed.it/> in your browser as an alternative option to view binary files.